Package 'ML.Classification.FeatureSelection'

May 7, 2026

Title Machine Learning Classification and Feature Selection Pipeline

Version 0.0.0.9000

Description Provides an R interface to a Python-based machine learning pipeline for binary and multiclass classification with bootstrap resampling, supervised feature filtering, model comparison, classification performance summaries, variable selection summaries, and feature stability visualization. The package uses reticulate to manage Python dependencies and run the bundled Python pipeline from R.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 8.0.0

Imports reticulate

Contents

check_python_setup 1
generateTestData 2
makePipelineDataFrame 3
run_ml_pipeline 3
use_ml_python_requirements 6

Index 7

`check_python_setup` *Check Python setup for the machine-learning pipeline*

Description

This function prints the active Python configuration and checks whether the Python modules required by the bundled machine-learning pipeline are available.

Usage

```
check_python_setup()
```

Details

The required modules include `numpy`, `pandas`, `sklearn`, `statsmodels`, `scipy`, `matplotlib`, and `seaborn`. The modules `lightgbm` and `tqdm` are treated as optional because the Python pipeline can skip LightGBM if it is not installed.

Value

Invisibly returns a data frame with module availability.

Examples

```
## Not run:
library(reticulate)
use_python("C:/Users/llm/anaconda3/python.exe", required = TRUE)

check_python_setup()

## End(Not run)
```

`generateTestData` *Generate simple two-view classification data for package testing*

Description

This function creates simulated two-view data with either binary or multiclass categorical outcomes. It is designed only for testing the reticulate-based ML pipeline package.

Usage

```
generateTestData(
  myseed = 1234L,
  n1 = 50L,
  n2 = 50L,
  n3 = NULL,
  p1 = 20L,
  p2 = 20L,
  n_signal = 5L,
  signal_strength = 1,
  test_fraction = 0.3,
  nReplicate = 1L
)
```

Arguments

`myseed`	Integer seed.
`n1`	Number of samples in class 1.
`n2`	Number of samples in class 2.
`n3`	Optional number of samples in class 3. If NULL, binary data are generated.
`p1`	Number of variables in view 1.

p2	Number of variables in view 2.
n_signal	Number of signal variables per view.
signal_strength	Mean shift for signal variables.
test_fraction	Fraction of samples used as test data.
nReplicate	Number of simulated replicates.

Value

A list with TrainData and TestData. Each replicate contains X and Y.

---

makePipelineDataFrame *Convert generated two-view data to a flat data frame*

Description

This converts the output of generateTestData() into the format required by run_ml_pipeline().

Usage

```
makePipelineDataFrame(mydata, replicate_id = 1L, use_test_data = FALSE)
```

Arguments

mydata	Output from generateTestData().
replicate_id	Which replicate to use.
use_test_data	Logical; if FALSE, uses TrainData. If TRUE, uses TestData.

Value

A data frame with RID, features, and outcome y.

---

run_ml_pipeline *Run the Python machine-learning classification pipeline*

Description

Runs the bundled Python machine-learning pipeline from R using `reticulate::source_python()`. The pipeline supports binary and multiclass classification and returns classification summaries, variable-selection summaries, bootstrap-level results, plot data, heatmap data, and label mappings.

Usage

```
run_ml_pipeline(
  training_data,
  outcome_col,
  demo_cols = NULL,
  id_cols = NULL,
  n_bootstrap = 100,
  pvalue_threshold = 0.05,
  min_features_after_filter = 5,
  n_iter_search = 20,
  random_state = 10,
  n_jobs = -1,
  make_performance_plot = TRUE,
  save_plot_path = NULL,
  show_plot = FALSE,
  plot_title = NULL,
  verbose = TRUE,
  progress_bar = TRUE,
  print_every_bootstrap = 1,
  make_stability_heatmap = TRUE,
  stability_heatmap_path = NULL,
  show_stability_heatmap = FALSE,
  heatmap_top_n_features = 30,
  heatmap_title = NULL
)
```

Arguments

`training_data`	A data frame containing the training dataset.
`outcome_col`	Character string giving the outcome column name.
`demo_cols`	Optional character vector of demographic covariates used in the supervised filtering step. These columns are not treated as model features in the final variable-selection summary.
`id_cols`	Optional character vector of ID columns to exclude from model fitting.
`n_bootstrap`	Integer number of bootstrap iterations.
`pvalue_threshold`	Numeric p-value threshold used in supervised filtering.
`min_features_after_filter`	Integer minimum number of features retained after supervised filtering.
`n_iter_search`	Integer number of random-search iterations for hyperparameter tuning.
`random_state`	Integer random seed passed to the Python pipeline.
`n_jobs`	Integer number of parallel jobs used by scikit-learn. For memory-limited environments such as ShinyApps.io, use `1` or `2`.
`make_performance_plot`	Logical; whether to create a classification performance plot.
`save_plot_path`	Optional file path for saving the classification performance plot.
`show_plot`	Logical; whether to display the performance plot.
`plot_title`	Optional character string for the performance plot title.

`verbose` Logical; whether to print progress messages.

`progress_bar` Logical; whether to show a Python progress bar if `tqdm` is available.

`print_every_bootstrap` Integer frequency for printing bootstrap progress updates.

`make_stability_heatmap` Logical; whether to create a feature-stability heatmap.

`stability_heatmap_path` Optional file path for saving the stability heatmap.

`show_stability_heatmap` Logical; whether to display the heatmap.

`heatmap_top_n_features` Integer number of top features to show in the stability heatmap.

`heatmap_title` Optional character string for the heatmap title.

Value

A list returned by the Python pipeline. Common elements include:

classification_summary Per-model performance summary.

variable_selection_summary Feature-ranking and selection-frequency table.

variable_selection_top_features Top selected features filtered by frequency.

best_bootstrap_variable_sets Selected features from the best bootstrap for each model.

bootstrap_results Bootstrap-level model results.

performance_plot_data Wide-format data used for the performance plot.

performance_plot_long Long-format data used for the performance plot.

stability_heatmap_data Feature-by-model stability table.

label_mapping Mapping from original outcome classes to encoded labels.

Examples

```
## Not run:
library(reticulate)
use_python("...", required = TRUE)

df <- read.csv("MCI-Convert-training.csv")

res <- run_ml_pipeline(
  training_data = df,
  outcome_col = "y",
  demo_cols = c("AGE", "PTGENDER", "APOE4"),
  id_cols = c("RID"),
  n_bootstrap = 10,
  n_jobs = 1,
  save_plot_path = "classification.png",
  stability_heatmap_path = "feature_stability_heatmap.png"
)

res$classification_summary
res$variable_selection_summary
res$bootstrap_results

## End(Not run)
```

use_ml_python_requirements *Declare Python dependencies for the machine-learning pipeline*

Description

Declares the Python packages and versions required by the bundled Python pipeline. This should be called before Python is initialized by reticulate.

Usage

```
use_ml_python_requirements()
```

Value

Invisibly returns NULL.

Examples

```
## Not run:
use_ml_python_requirements()

## End(Not run)
```

Index

check_python_setup, 1

generateTestData, 2

makePipelineDataFrame, 3

reticulate::source_python(), *3*
run_ml_pipeline, 3

use_ml_python_requirements, 6